

Reviewer Pack — SOS Hierarchy Integrality Gap for Random CSPs

Entry 90b9c49aa839 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-29 02:17:12 UTC

SOS Hierarchy Integrality Gap for Random CSPs

Entry ID: 90b9c49aa839 Verdict: INCONCLUSIVE Recorded: 2026-04-29 02:17:12 UTC

1. Conjecture

Field A (mathematical branch): Sum-of-Squares hierarchy **Field B** (complexity object): Constraint Satisfaction Problems (CSPs)

Statement:

For any constant $\varepsilon > 0$, there exists a CSP instance with n variables and $m = \Theta(n)$ constraints such that the SOS hierarchy requires $\Omega(n^{1-\varepsilon})$ levels to achieve an integrality gap of $1 - \varepsilon$ for its relaxation.

Rationale (proposer’s reasoning):

This conjecture builds on the SOS hierarchy’s known limitations in handling random CSPs, where the hierarchy’s ability to capture exact solutions scales sublinearly with the problem size. It ties to the approach’s target of analyzing refutation sizes and integrality gaps, avoiding relativization barriers by focusing on structural properties of random CSP instances.

Taxonomy category: SOS_HIERARCHY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0279ff991aa35750

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below the pivot
        factor = Fraction(1, A[i][i])
        for j in range(i+1, n):
            A[j][i] *= factor

        # Eliminate above the pivot
        for j in range(i):
            factor = A[j][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return [sum(row) for row in A]

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 30
    m = n

    # Generate a random 3-CNF formula
    clauses = []
    for _ in range(m):
        literals = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
        random.shuffle(literals)
        clauses.append(literals[:3])

    # Convert to matrix form
```

```

A = [[0] * n for _ in range(n)]
for clause in clauses:
    for literal in clause:
        var = abs(literal) - 1
        if literal > 0:
            A[var][var] += 1
        else:
            A[var][var] -= 1

# Compute the rank of the matrix
rank = gaussian_elimination(A)

# Check if the hierarchy requires  $\Omega(n^{(1-\epsilon)})$  levels
 $\epsilon$  = Fraction(0.1) # Constant  $\epsilon > 0$ 
required_levels = n ** (1 -  $\epsilon$ )
conjecture_holds = rank >= required_levels

return {
    "metric_name": "SOS Hierarchy Levels",
    "metric_value": rank,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else "mapping_undefined"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_566ald2d.py", line 85, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_566ald2d.py", line 64, in run_trial
    rank = gaussian_elimination(A)
           ^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_566ald2d.py", line 29, in gaussian_elim
    factor = Fraction(1, A[i][i])
             ^^^^^^^^^^^^^^^^^
File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
ZeroDivisionError: Fraction(1, 0)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with ZeroDivisionError, preventing data collection. Support condition cannot be evaluated without successful trials. | next: Debug gaussian_elimination() to handle zero pivots, perhaps by adding numerical tolerance or matrix preconditioning

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	102722
2	preregistration	ollama_remote	qwen3:8b	0	0	34561
3	novelty	ollama_remote	qwen3:8b	0	0	23910
4	novelty	ollama_remote	qwen3:8b	0	0	23635
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15550
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11281
7	judge	ollama_remote	qwen3:8b	0	0	18290

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 229949 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/90b9c49aa839.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/90b9c49aa839.jsonl for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/90b9c49aa839.tar.gz` (if generated)